

Android 应用抗加固分析： 代码混淆、加壳与环境检测技术的检测框架设计

洪奕迅 蒋城昊 项康

Abstract

在当前互联网生态中，Android 操作系统在移动端市场中占据着绝对的主导地位，但这也使其不可避免地遭受严峻的安全威胁。为了规避基于静态特征扫描和动态沙箱分析的安全检测机制，恶意软件开发者广泛采用代码混淆、运行时加壳以及环境感知检测等抗分析技术。相应地，合法应用开发者也使用同类技术来保护知识产权。这使得加固技术本身呈现出一种中性特征，同时也意味着仅检测“是否被加固”已不足以支撑安全分析决策。

基于上述情况以及课程 Final Project “Android 应用抗加固分析”选题的任务要求，本文系统综述了近年来发表于 IEEE S&P、USENIX Security、NDSS、ACM CCS 等安全会议以及 TIFS、TSE 等期刊上的代表性工作。本文首先阐述了字节码层代码混淆、运行时加壳和环境感知检测这三个维度的加固技术；进而对现有对抗上述加固技术的方法进行了深入分析和比较，包括基于静态特征的机器学习检测、基于动态插桩的脱壳还原、以及基于硬件的无侵入式跟踪等方法。

在此基础上，本文进一步介绍了现有工作在技术识别粒度、静态与动态的权衡、可解释性与可部署性等方面存在的显著问题。最后，本文提出了一个 APK 加固技术检测框架 *HardenInspector*，并详细论述了其解析层、特征层、决策层的核心架构，以及作为可选扩展的动态验证层。本文的目标是为期末阶段的工作提供扎实的理论基础与可行的实现路线。

1 引言

1.1 研究背景

Android 操作系统是当前部署最广泛的移动计算平台之一。NDSS 2022 的研究指出，Android 在移动操作系统市场中占据超过 70% 的份额，庞大的终端规模和应用生态使其长期成为移动恶意软件和逆向分析对抗的重要目标 [9]。Android 恶意软件已从早期的短信扣费、隐私窃取等简单形态，演变为具备多层对抗能力的复杂威胁。它们能够主动识别分析环境、隐藏核心载荷，并在检测到沙箱或调试器时伪装为正常行为 [9]。

大量学术研究表明，Android 恶意软件正在持续采用混淆、动态加载和运行时规避等对抗机制。TIRO 在 2,000 个来自 Virus-Total 的 Android 恶意样本中发现，运行时混淆技术出现在 80% 的样本中 [17]；PackerGrind 和 Happer 等工作也表明，现代 Android 加壳器已能够结合动态解密、方法级代码修改和反分析机制来阻碍传统静态与动态分析 [19, 20]。因此，传统的基于签名的检测方法（如反病毒引擎的哈希匹配和 YARA 规则）在面对混淆和加壳保护时效能会显著下降，因为一个样本经过标识符重命名、字符串加密或动态加载后，其静态特征可能与原始样本产生明显差异。

在此背景下，学术界和工业界投入了大量资源研发自动化检测系统。早期工作主要依赖静态特征提取与机器学习分类 [2, 12]，通过应用包 (APK) 中提取权限、API 调用、组件声明等特征来构建分类模型。Drebin [2] 在 2014 年的 NDSS 上证明了

使用线性 SVM 在大规模数据集上高效检测恶意软件的可行性，在 123,453 个应用上实现了 94% 的检测率。MaMaDroid [12] 进一步通过 API 抽象和 Markov 链建模展现了其对 API 版本变化的天然鲁棒性，在训练后两年仍保持 75% 的 F-measure。

然而，随着混淆和加壳技术的广泛部署，静态特征的有效性被严重削弱 [6, 11]。CorDroid [6] 的实验揭示了一个关键事实：不同混淆技术对不同静态特征会产生互补性破坏，例如标识符重命名几乎不改变 Dalvik 操作码序列，但严重扰乱调用图，反射调用却相反。这一特性既造成了静态检测的挑战，因为任何单一特征都可能被针对性绕过，也指向了潜在的解决方案，即多特征融合。

但加固技术本身并非恶意软件的专属工具。从工业实践的角度看，加固技术已渗透到 Android 应用生态的各个角落。合法的金融类应用使用加壳保护支付逻辑；游戏开发商使用混淆防止外挂和盗版；企业移动管理 (EMM) 方案使用环境检测确保设备合规性。与此同时，恶意软件利用同样的技术规避检测，比如 APT 组织开发的 Android 间谍软件就集成了多层加固保护。这种双面性使得加固检测成为一个比传统恶意软件检测更为微妙的问题：检测的目标不应该是将使用加固技术与恶意划上等号，而应该是识别加固技术的类型、深度和组合来为后续的安全决策提供信息基础。这一转变为本文的研究提供了核心动机。

1.2 课程任务

本文面向《系统安全》课程 Final Project 相关选题的任务要求，将整体目标拆分为三个子任务：(1) 了解常见的代码混淆、加壳、环境检测等技术及其实现原理；(2) 通过小规模复现实验验证环境感知规避在模拟器场景中的表现；(3) 实现一个以静态规则为核心、能够输出证据链的检测工具原型。

本篇综述具体完成了以下工作：(a) 对实现 Android 加固的三类技术的实现原理和可观测特征进行了系统分析；(b) 对现有研究方法按静态 ML 检测、动态脱壳与分析、环境对抗、程序分析基础设施四个方向进行了系统概括和能力比较；(c) 分析了现有工作在技术识别粒度、静态动态权衡、可解释性等方面存在的不足；(d) 提出了 *HardenInspector* 框架的设计方案，包括其架构、特征体系、决策策略以及可选的动态验证层，并给出了评估计划和实现路线。

1.3 Android 加固技术概述

Android 应用的加固技术可以按其作用层面分为三大类：

- (1) **代码混淆**：在字节码或源码层面扰乱程序的符号信息、控制流和数据引用，使逆向分析难以理解程序逻辑，同时保持语义等价。常见技术包括标识符重命名、控制流平坦化、字符串加密、反射调用替换、垃圾代码注入等 [1, 6]。
- (2) **运行时加壳**：将原始 Dalvik 可执行文件 (DEX) 或敏感方法加密存储，仅在运行时解密并动态加载。加壳器通常还集成反调试、反 DBI 等机制以阻止分析工具的运行时提取 [19, 20, 23]。
- (3) **环境感知检测**：在运行时检测模拟器、调试器、动态插桩框架等环境特征，一旦发现即终止或隐藏恶意行为。此类检

测已从简单的 Build.FINGERPRINT 比对演化为多维度的环境指纹匹配 [9, 13]。

这三类技术在真实恶意样本中常以协同方式出现: 加壳器外层以环境检测拒绝沙箱执行, 脱壳后的 DEX 仍施加多层混淆, 而最敏感的逻辑往往通过反射或 JNI 调用进一步隐匿。这种结构使得任何单一维度的检测器都极易被绕过。

2 Android 抗分析技术: 原理与可观测特征

本节对三类加固技术的实现原理进行深入分析, 重点关注这些技术留下的可观测特征, 即可以被静态或动态分析方法捕捉到的指纹。

2.1 字节码层代码混淆

2.1.1 标识符混淆. 标识符混淆是最基础也是最为广泛使用的混淆技术, 通过将类名、方法名、字段名替换为无意义的短字符串来消除语义信息。ProGuard 采用字母序重命名 (a, b, c, ...), 商业工具如 Allatori 和 DexGuard 则使用更复杂的策略 (包括不可打印字符、Unicode 同形字等)。

从检测角度来看, 标识符混淆在字节码层面留下了显著的可量化指纹: (1) 短标识符在常量池中的统计分布异常: 正常应用中标识符长度通常服从正态分布, 均值为 8-12 字符, 而混淆后的标识符大量集中在 1-3 字符长度; (2) 包名中出现非标准层次结构; (3) AndroidManifest 中组件类名与 DEX 中实际存在的类名不匹配, 这是重打包工具的常见副作用 [15]。

CorDroid 的实验表明, 标识符混淆对基于包名的特征如 MaMaDroid [12] 影响最大, 但几乎不改变 Dalvik 指令序列 [6]。所以在后续特征设计时, 我们应当同时采集多类特征来支持不同混淆技术的识别。

2.1.2 控制流混淆. 控制流混淆在不改变程序语义的前提下扰乱控制流图的结构。主要子类包括:

控制流平坦化 (Control Flow Flattening): 将函数的所有基本块扁平化为一个大型 switch-case 结构, 由分发变量决定跳转目标。其可观测特征包括: (1) 方法入口处出现典型的分发循环模式; (2) 每个基本块结尾的 goto 指令指向分发器的头部; (3) 基本块间的自然前后关系被破坏。Obfuscapk [1] 实现了基于 Smali 级别的控制流平坦化。

调用间接化 (Call Indirection): 将原有的直接调用 A→B 改写为 A→C→B, 在函数调用图中引入额外的中间节点, 破坏了调用图结构的简洁性。DroidSieve [15] 和 RevealDroid [7] 通过分析 FCG 的结构度量来捕捉此类异常。

不透明谓词注入: 在控制流中引入恒真或恒假的条件分支, 增加控制流图的复杂度。在字节码层面表现为额外的 if-* 指令和不必要的分支目标。

2.1.3 数据混淆. 数据混淆主要包括以下类别:

字符串加密: 将 DEX 中的所有字符串常量加密存储, 运行时通过解密函数还原。可观测特征包括: (1) 常量池中的字符串项被替换为加密后的二进制数据; (2) <clinit> 静态初始化块中出现特征性的解密逻辑; (3) 大量字符串操作函数的参数来自长生命周期数组。

反射调用替换: 将原有的直接方法调用替换为 java.lang.reflect.Method.invoke() 的动态反射调用。这导致: (1) 方法调用在调用图中形成密集的星型结构, 即大量调用边指向 Method.invoke, 但真实目标不可解析; (2) 字符串常量池中出现大量方法签名字符串; (3) 类的加载模式出现异常。Gao 等人在 CorDroid [6] 中发现反射调用对函数调用图的影响最大, 可

使有用边减少 60% 以上, 但几乎不改变 bytecode 级别的操作码分布。

2.1.4 代码布局混淆. 代码布局混淆主要包括以下类别:

垃圾代码注入: 在方法中插入语义无关的指令, 如无用的算术运算、从不执行的 goto 跳转、永不使用的变量赋值等。可观测特征包括: (1) 方法中的 NOP 类指令密度异常增高; (2) const-string 后跟从未使用的字符串常量; (3) 基本块中指令数量呈现不自然的分布。

指令重排序: 通过 goto 指令人工打乱基本块的自然顺序。该技术保留了方法内部的原始控制流结构, 但改变了块在字节码文件中的物理排列顺序。其检测难度较高, 因为合法的编译器优化 (如 Dalvik 的 dexopt) 也可能产生类似的重排效果。

2.2 运行时加壳

2.2.1 加壳器的工作流水线. 图 1 描绘了典型 Android 加壳器的工作流水线。现代商业加壳器的保护机制已从简单的整体 DEX 加密演变为包含 “加密 → 反调试 → 环境检测 → Native 化” 的多阶段管线 [19]。

2.2.2 DEX 隐藏策略. 现代加壳器采用的核心 DEX 隐藏策略可分为三类:

(1) **整体加密、动态释放:** 原始 DEX 以加密形式存放于 APK 的资源目录、lib/ 中的 so 库数据段、或加壳器自定义的文件格式中。在 Application.onCreate() 或 attachBaseContext() 阶段, 加壳器将解密后的 DEX 通过反射调用 DexClassLoader 或直接操作 ClassLoader 的 DexPathList 字段注入运行时期。此策略的静态指纹包括: (1) assets/ 中出现疑似加密 DEX 的文件; (2) AndroidManifest.xml 中声明的 Application 类被替换为壳程序类; (3) Native 库中包含 DEX 解析相关的符号。

(2) **方法级动态修改 (FmTmF 模式):** Happer [20] 发现部分高级加壳器采用 “填充-执行-抹除” 的细粒度策略: 仅在 ART 即将执行某个方法前填充其字节码, 执行完并立即将内存中的指令区域清零。这种策略使整体 DEX 在任意时刻都不在内存中完整存在, 极大地增加了动态转储的难度。

(3) **Native 化 (JNI 替代):** 将关键的 Java 方法整体替换为 JNI 调用, 使逻辑不再以 DEX 字节码形式存在。商业加壳器如 Qihoo 360 加固 (v3 以上版本)、Baidu 加固正是使用此策略。静态指纹包括: (1) DEX 中存在大量 native 方法声明; (2) 对应的 .so 库中包含 JNI_OnLoad 函数; (3) 关键敏感 API 的调用来源均指向 native 方法。

2.2.3 反分析机制. 几乎所有的商业加壳器都内建了针对分析工具的反制措施。PackerGrind [19] 与 Happer [20] 独立总结出了相似的观察:

反调试: 通过 ptrace(PTRACE_TRACEME) 自挂载阻止其他调试器附加; 周期性检查 /proc/self/status 中的 TracerPid 字段; 扫描内存中的断点指令。

反 DBI 框架: 扫描 /proc/self/maps 中是否加载了已知 DBI 库 (libvalgrind*.so, libfrida-gadget*.so, libxposed*.so); 检测特定页面的权限组合。

反模拟器: 除传统的 Build.FINGERPRINT 和 ro.kernel.qemu 检测外, 现代加壳器还利用传感器缺失、电池状态异常、以及 CPU 指令执行时间差异等特征来识别模拟器 [9]。

2.3 环境感知检测

2.3.1 从“红丸检测”到多维指纹. 传统的环境检测 (“红丸”检测, Red Pill) 依赖固定的系统属性字符串匹配。然而, Kondracki

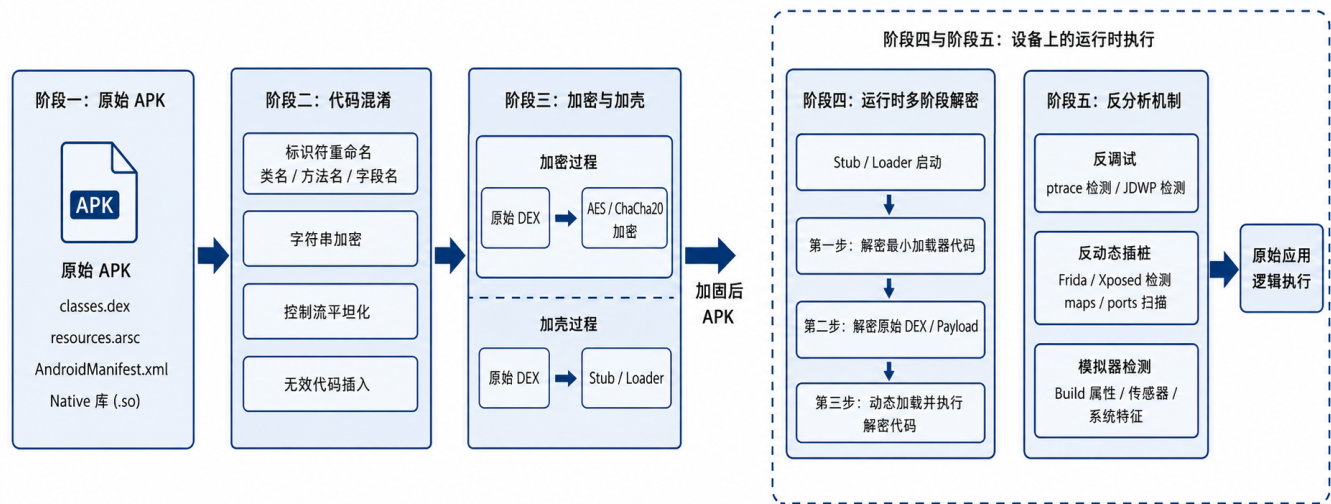


Figure 1: 典型 Android 加壳器的保护流水线：原始 DEX 首先经过混淆处理，随后加密嵌入加壳后的 APK 中；运行时经多阶段解密释放，同时伴随反调试与环境检测机制。

等人的研究 [9] 揭示了现代环境检测的范式转变：攻击者已从针对特定模拟器的已知字符串匹配, 转向利用普通 Android API 观测设备的多维特征并进行组合判定。

他们将可用于环境检测的特征归纳为三个维度：(1) **使用磨损特征**——照片库数量、联系人数量、通话记录、Wi-Fi 配置数等反映真实用户长期使用痕迹的数据；(2) **设备配置特征**——自动同步时间设置、屏幕亮度模式、是否启用开发者选项；(3) **硬件指纹特征**——制造商与型号的组合一致性、各类传感器的存在性。表 1 给出了每一类别中最具区分力的特征子集。

Table 1: 环境感知检测中的高区分力特征（整理自 [9]）

类别	检测特征	区分力
使用磨损	照片数量、最近拍照时间	高
	联系人数量、星标联系人	高
设备配置	已安装包数量	中
	自动同步时间	高
	声音模式、开发者选项状态	中
硬件指纹	设备 Uptime	高
	制造商 + 型号组合一致性	高
	传感器存在性向量	高

2.3.2 环境检测的代码实现模式. 从字节码层面分析，环境检测代码通常呈现出以下可识别模式：(1) 在 Application.<clinit> 或早期生命周期回调中集中调用 System.getProperty()、Context.getSystemService()、Settings.System.get*() 等 API；(2) 调用结果与硬编码常量比较后触发控制流分支，一个分支通向正常行为，另一分支通向休眠或退出；(3) 使用 Class.forName() 探测特定类的存在性 (如针对 com.frida.react.SessionManager 的探测可用于识别 Frida 注入)。这些模式是识别自动化环境检测意图的关键。

3 相关工作

本节将现有研究分为四个主要方向进行阐述。

3.1 基于静态特征的机器学习检测

Drebin (NDSS 2014) [2] 是该领域的里程碑式工作。它将 Android 应用的静态特征 (权限、API 调用、Intent 等) 嵌入高维向量空间，使用线性 SVM 进行恶意/良性分类，在超过 12 万个应用上实现了 94% 的检测率。Drebin 的贡献在于证明了无需动态执行即可实现高效检测的可行性，但其特征设计未考虑混淆对抗，简单的标识符重命名也会使基于 API 调用频率的特征产生显著漂移。

MaMaDroid (NDSS 2017) [12] 提出了一个对 API 变化具有天然鲁棒性的检测框架。其核心思路是将 API 调用抽象到包或家族级别，基于抽象后的调用序列构建 Markov 链作为行为模型。这种抽象策略使特征维度可控 (包模式下约 11K，家族模式下仅 9 类)，同时降低了 API 版本更新引起的特征失效风险。然而，MaMaDroid 对字节码级别的混淆 (如调用间接化、控制流平坦化) 仍缺乏防护。混淆会改变调用图中的节点连接模式，进而影响 Markov 转移概率。

DroidSieve (CODASPY 2017) [15] 明确将“混淆弹性”作为设计目标。其创新在于提出了资源中心特征，即从 APK 的 assets/ 目录中提取 ELF 文件、证书信息、incognito app 结构等特征，而这些特征通常不受字节码混淆的影响。同时，它利用混淆本身引入的人为痕迹作为检测信号，例如混淆器插入的特定类加载模式、非标准的证书时间戳等特征。在 100K+ 样本上实现了 99.82% 的检测准确率和零误报。

RevealDroid (ICSE 2018/TSE 2020) [7] 进一步推进了轻量级混淆检测的方向。它选用三类轻量级特征：分类 API 使用统计、基于反射的特征、以及 Native 二进制特征。在 54K 样本上达到 98% 的检测准确率和 95% 的家族识别准确率。

CorDroid (TIFS 2023) [6] 是目前在混淆弹性方面最为系统的工作之一。它使用双流模型 (E-SFCG 经 GCN 编码语义级特征和 OMM 经 EfficientNet 编码语法级特征) 在 14 种混淆变换下保持高 F1。但 CorDroid 的目标是恶意检测而非混淆识别, 其权重向量不能直接转换为混淆技术标签。

MalScan (ASE 2019) [18] 另辟蹊径, 将函数调用图建模为社会网络, 通过计算敏感 API 在网络中的中心性度量来表征应用的行为特征。这种基于图拓扑的方法具有较好的扩展性。

3.2 基于动态分析的脱壳与行为监控

DroidScope (USENIX Security 2012) [21] 是 Android 动态恶意软件分析的先驱性平台。它基于 QEMU 虚拟机, 通过虚拟机自省 (VMI) 技术同时重建 OS 层和 Dalvik 层的语义视图, 提供硬件级、OS 级和 Dalvik 级三层 API 供分析工具调用。DroidScope 从虚拟机外部透明监控目标系统内部状态的这一核心设计理念深刻影响了后续几乎所有的 unpacking 系统。基于 DroidScope, 研究者开发了指令追踪、API 监控和信息流追踪等分析插件。然而, DroidScope 仅支持 Dalvik 虚拟机 (Android 4.4 及之前版本), 对 Android 5.0 之后的 ART 运行时缺乏支持, 这一限制催生了后续的 DroidUnpack 和 PackerGrind 等工作。

DexHunter (ESORICS 2015) [23] 首次系统研究了商业 Android 加壳器的技术特征, 并提出了一种通用的 DEX 提取方法。其核心思路是利用 Android 类加载过程的必经路径, 即在 ART/DVM 的关键函数 (如 `DexFile::OpenMemory`、`DefineClass`、`LoadMethod`) 处挂钩, 在加壳器将解密后的 DEX 数据送入类加载器时截获数据。DexHunter 是首批同时支持 DVM 和 ART 双运行时的 unpacking 工具之一, 其通用性设计使其能够处理当时市面上的主流加壳器, 而无需针对每种加壳器开发专门的提取逻辑。然而, DexHunter 对“动态代码修改”, 即加壳器在类加载后继续修改字节码的行为的处理能力有限。

AppSpear (RAID 2015) [22] 采用“字节码解密 + DEX 重组”的方法实现通用脱壳。与 DexHunter 在类加载阶段截获数据不同, AppSpear 直接插桩 Dalvik VM, 从 Dalvik Data Struct 中收集解密后的字节码信息, 然后通过精细化的重组过程生成新的有效 DEX 文件。其独特优势在于: (1) 能够处理 DEX 文件结构被故意破坏的情况 (如加壳器修改了 DEX 头部的 checksum 或 signature 字段); (2) 对 37,688 个 Android 恶意样本进行了大规模的加壳普及率统计, 为社区提供了关于加壳技术流行度的量化数据。

TIRO (USENIX Security 2018) [17] 从一个不同的角度切入去混淆问题。它提出了 Target-Instrument-Run-Observe 的四步框架, 能够统一处理传统混淆 (语言层面的标识符重命名、反射等) 和运行时混淆 (runtime-based obfuscation)。运行时混淆是 TIRO 引入的一个重要概念, 它是指恶意软件通过颠覆 ART/DVM 运行时本身 (如修改虚函数表、Hook JNI 接口、利用 ART 内部未公开的 API 行为) 来隐藏真实行为。TIRO 在 2,000 个来自 VirusTotal 的样本中发现 80% 使用了运行时混淆, 揭示了这种此前被忽视的威胁维度。TIRO 证明了这种针对性执行, 即通过静态分析识别混淆点并确定触发输入然后在受控环境中仅执行这些混淆点的执行方式, 比全量动态执行更加高效。

DroidUnpack (NDSS 2018) [5] 基于全系统模拟的方法实现了加壳行为的最底层监控。它通过从 QEMU 外部自省来賓 Android 系统, 重建 OS 和 ART 两层语义视图, 能够可靠地捕获先写后执行的加壳行为模式, 而这是所有加壳器在本质上无法

避免的特征, 因为加壳器必须在某一时刻将隐藏的代码写入内存并随后执行它。DroidUnpack 的一个重要发现是, 现有的三种 unpacker (基于 ART/DVM 修改的、基于 DBI 框架的、基于模拟器的) 在处理高级加壳器时都存在严重的覆盖面不足这一问题, 因为它们无法处理多层次的代码修改和 Native 层混淆。

DexLeg0 (DSN 2018) [14] 提出了“可重组的字节码提取” (re-assembleable bytecode extraction) 的新思路。它利用 JIT 收集机制在指令级别采集执行过的字节码和数据, 然后离线将这些离散的指令片段重组为有效的 DEX 文件。JIT 收集的优势在于粒度极细, 可以捕获到仅在特定执行路径上出现的字节码, 包括自修改代码产生的临时指令。其重组算法的关键挑战是将离散的指令片段正确还原为完整的类、方法和字段结构, 而这需要设计一个能够处理缺失依赖和跨方法引用一致性的重组算法。

PackerGrind (TSE 2022) [19] 与 **Happer (S&P 2021)** [20] 代表了当前 unpacking 技术的两种前沿路线。PackerGrind 基于 Valgrind DBI 框架在三层 (Runtime/System/Instruction) 上挂钩 ART 关键函数 (`DexFile::OpenMemory`、`DefineClass`、`LoadMethod`、`ExecuteSwitchImpl/ExecuteGotoImpl`), 在不同 Dex 数据采集点动态转储字节码并迭代执行, 直至所有方法均含有效字节码。该方法可处理最复杂的 FmTmF 模式 (填充-执行-抹除-填充循环), 但仍受限于 Valgrind 自身的可检测性, 因为商业加壳器已广泛部署 Valgrind 指纹扫描。Happer 则利用 ARMv8 的 ETM (Embedded Trace Macrocell) 硬件追踪和 HBRK (Hardware Breakpoints) 实现完全无侵入的指令级追踪——追踪是硬件外置的, 加壳器在软件层面无法感知追踪器的存在。Happer 通过自定义的 BDL (Behavior Description Language) 描述待跟踪的加壳行为, 实现了对 T、FmT、TmF、FmTmF 四种基本加壳模式的全覆盖。而 Happer 的代价是对硬件的强依赖 (需要 ARMv8 + ETM + DSTREAM 调试探针), 这使其难以被大规模部署。

3.3 环境感知的攻防对抗

BareDroid (ACSAC 2015) [13] 观察到模拟器与真实设备之间存在根本性的差异, 其解决方案是直接裸机设备上进行分析, 通过快速恢复真机到系统初始状态实现了在真实设备上进行分析的可行性。

Andrubis (BADGERS 2014) [10] 是一个公开可用的大规模混合分析系统, 结合了静态分析与 Dalvik VM 及系统级的双层动态分析。截至 2014 年发表时, 已分析了超过 100 万个 Android 应用。

The Droid is in the Details (NDSS 2022) [9] 进一步扩展了环境检测的研究维度, 系统证明了 75% 的现有沙箱因设备身份谎报而可被识别。

3.4 程序分析基础设施

FlowDroid (PLDI 2014) [3] 是 Android 静态污点分析领域最具影响力的工作之一。它首次实现了完全上下文敏感、流敏感、字段敏感和对象敏感的 Android 生命周期感知污点分析, 并提出了 DroidBench 基准测试集。

Amandroid (CCS 2014) [16] 提供了一个通用的组件间数据流分析框架, 通过流敏感和上下文敏感的方式计算应用内所有对象的指向信息, 能够构建包含组件间通信边的过程间控制流图。

DroidSafe (NDSS 2015) [8] 进一步推进了 Android 静态分析的精度边界。通过对 AOSP 源码的分析建立了 Android API 和

运行时行为的精确模型。在独立红队注入的测试中检测率达到了 100%。

Obfuscapk (SoftwareX 2020) [1] 是一个开源的黑盒 Android 混淆工具, 支持包括控制流平坦化、字符串加密、资源混淆等多种技术, 其模块化架构有助于了解混淆技术实施细节。

PRAGuard 数据集 (Computers & Security 2015) [11] 由 Maiorca 等构建, 由对 MalGenome 和 Contagio Minidump 数据集施加七种不同混淆技术生成, 共包含 10,479 个样本, 为混淆弹性检测器的训练和评估提供了可控环境。

3.5 小结

表 2 系统总结了上述工作面向“加固技术识别”这一任务的能力覆盖情况。

Table 2: 相关工作面向加固技术识别的能力总结

工作	混淆	加壳	环境检测	解释性
Drebin [2]	△	×	×	中
MaMaDroid [12]	△	×	×	低
DroidSieve [15]	◦	△	×	低
RevealDroid [7]	◦	△	×	低
CorDroid [6]	◦	×	×	低
MalScan [18]	△	×	×	低
DexHunter [23]	×	•	×	中
AppSpear [22]	×	•	×	中
TIRO [17]	•	•	×	中
DroidUnpack [5]	×	•	△	中
PackerGrind [19]	△	•	△	中
Happer [20]	△	•	△	中
BareDroid [13]	×	×	•	低
Droid-is-in-Details [9]	×	×	•	高
本文原型目标	◦	◦	◦	高

• = 主要能力, ◦ = 部分覆盖, △ = 间接覆盖, × = 不适用。本文原型目标表示期末项目计划覆盖的静态检测维度, 并非已完成系统的实验结果。

3.6 工作脉络与演化趋势

上述工作呈现出几条清晰的演进脉络:

从特征工程到表示学习: Drebin [2] 的手工特征集开创了 Android 恶意软件 ML 检测的范式, 但其对混淆的脆弱性也迅速暴露。MaMaDroid [12] 的 API 抽象策略提供了一定程度的鲁棒性, 但仍无法应对字节码级混淆。DroidSieve [15] 和 RevealDroid [7] 通过引入资源层和 Native 层特征来绕过字节码混淆这。而 CorDroid [6] 的双流 GCN+EfficientNet 则代表了表示学习路线的最新进展。

从软件插桩到硬件辅助: DroidScope [21] 开启了虚拟化辅助的动态分析范式, 但软件级分析工具的可检测性始终是弱点。Happer [20] 通过 ARMv8 ETM 硬件追踪将分析变为完全外置的, 但硬件方案的成本反过来限制了其应用范围。这一矛盾揭示了本领域的根本困境: 最强大的分析能力往往伴随着最差的部署友好性。

从攻防分离到协同视角: 传统上, 环境检测研究 (如 BareDroid [13] 和 Droid-is-in-the-Details [9]) 与混淆/加壳研究是分离的。前者关注“如何让分析环境不被检测”, 后者关注“如何穿透混淆和加壳的保护”。但在真实恶意样本中, 这两者是

紧密耦合的——环境检测是加壳器保护链的第一道防线。HardenInspector 正是基于这一认识, 将三个维度统一到一个分析框架中, 实现了从“分离攻防”到“协同识别”的范式转换。

从学术研究到工业实践的鸿沟: 尽管学术界已提出诸多先进的检测和脱壳技术, 工业界的加固检测实践仍主要依赖 AV 引擎的签名匹配和 VirusTotal 的多引擎交叉验证。造成这一现象的原因是多方面的: (1) 学术工具通常假设特定的 Android 版本和运行环境 (如修改过的 ART、特定的 QEMU 版本), 而工业环境需要兼容广泛的设备型号和系统版本; (2) 学术方法的评估通常在大规模精选数据集上进行, 而工业场景面临的是每日数以万计的新样本; (3) 许多学术工具缺乏可维护的开源实现, 其复现和维护成本超出了大多数安全团队的资源预算。HardenInspector 的设计明确地试图弥合这一鸿沟, 他通过将核心分析逻辑限制为纯静态、与 Android 版本解耦的特征提取, 同时将动态执行作为可选的验证模块, 使其既能在学术实验环境中提供精确识别, 也能在工业预筛管道中以轻量级模式运行。

3.7 开源工具与数据生态

表 3 总结了本领域的关键开源工具与数据集。

Table 3: Android 安全分析关键开源工具与数据集

名称	类型	作用
Androguard [4]	逆向分析框架	DEX/Manifest 解析基础库
FlowDroid [3]	静态污点分析	调用图构建参考实现
Obfuscapk [1]	混淆工具	可控混淆样本生成工具
Frida	动态插桩框架	可选运行时复核工具
PRAGuard [11]	混淆数据集	混淆技术对照参考
AndroZoo	APK 仓库	大规模良性样本来源
VirusTotal	多引擎扫描	恶意标签验证来源

4 现有方法的局限

本节从识别与检测、静态与动态、可解释性与可部署性三个角度出发, 逐一梳理现有方法的问题。

4.1 从识别到检测

现有绝大多数 Android 恶意软件检测研究的设计目标是二分类或多分类 (家族归属), 而非技术识别。例如 Drebin [2] 的 SVM 超平面、MaMaDroid [12] 的 Markov 转移概率矩阵, 这些模型的内在结构无法直接映射到技术标签。

CorDroid [6] 虽然明确考虑了混淆的影响, 但其输出仍然是一个二分类置信度。这种识别粒度的缺失导致了下面两个问题: (1) 分析者无法根据模型输出了解应用的加固技术栈; (2) 当新混淆技术出现时, 无法快速定位模型失效的原因。

4.2 静态与动态的权衡

现有 unpacking 系统的成功建立在动态执行的基础上, 它们需要在实际运行环境中让加壳器释放 DEX 数据。这一策略的本质矛盾在于加壳器的环境检测机制恰恰是为了阻止在分析环境中运行。解决方法是要么提升模拟器的透明性, 如 BareDroid [13], 要么使用无法被检测的硬件级追踪, 如 Happer [20], 但两者都以成本和可部署性为代价。

一个更为现实的思路是, 加壳器的识别并不总是需要完整的脱壳。许多加壳器在 APK 的静态层面留下了足够的指纹, 可以

通过轻量级的静态扫描加以识别。将动态分析保留为进一步的判断手段而非必要环节,可以在保持识别能力的同时大幅降低分析成本。

4.3 可解释性与可部署性

现有 ML 方法的输出通常是一个概率值或分类标签,缺乏对决策过程的解释。这在安全分析场景中是一个很严重的缺陷,因为分析者无法验证结果的正确性,也无法将检测结果用作进一步分析的输入。在可部署性方面,基于硬件的方案和基于真机的方案都受限于规模和成本。

4.4 小结

综合以上分析,HardenInspector 框架明确区别于现有工作的定位是:(1)任务目标:识别加固技术本身,而非判定恶意性或还原源码;(2)分析方法:以静态规则为主,动态复核为可选扩展;(3)输出形式:输出多标签结果以及可追溯的证据链;(4)部署模式:采用规则与解析逻辑分离的模块化设计,支持普通笔记本运行。

5 HardenInspector: 加固技术检测框架设计

本节详细阐述 HardenInspector 的架构设计与关键技术方案。

5.1 设计原则

HardenInspector 的设计遵循五项原则:(1)**综合性**:同时覆盖混淆、加壳与环境检测三个维度;(2)**可解释性**:对每个技术标签给出触发证据链;(3)**轻量性**:以静态分析为主,在普通笔记本上完成扫描;(4)**可扩展性**:检测规则与代码主体解耦;(5)**模块化**:四层架构各层独立,层间通过明确定义的接口通信。

5.2 整体架构

图 2 展示了 HardenInspector 的架构概览。期末 MVP 的主路径为:输入 APK → 解析层解构 → 特征层提取多维特征 → 决策层进行规则匹配与风险分级 → 输出检测报告。图中的动态验证层如果出现,应理解为虚线扩展路径:它用于少量样本的运行时复核,不属于核心交付和主要评估对象。

5.3 解析层

解析层负责将输入的 APK 文件解析为后续特征提取所需的结构化中间表示。具体步骤:

- (1) **APK 解包**:提取 AndroidManifest.xml、classes*.dex、lib/<abi>/、assets/、META-INF/。
- (2) **DEX 解析**:基于 Androguard [4] 解析每个 DEX 文件,提取常量池、类定义、字节码指令序列。
- (3) **SO 解析**:对每个 Native 库进行 ELF 格式解析,提取导出符号表、.rodata 段字符串、文件哈希。
- (4) **Manifest 解析**:提取权限声明、四大组件声明、Application 类名、Intent Filter、硬件依赖声明。

5.4 特征层

特征层是 HardenInspector 的核心,从解析层的中间表示中提取三个维度的特征向量。

5.4.1 加壳特征.

- (1) **Native 库指纹**:对每个.so 文件计算文件名与已知加壳器 SO 库的精确匹配和编辑距离(如 libjiagu.so→360

加固、libsecexe.so→梆梆、libDexHelper.so→爱加密、libtup.so→腾讯乐固)。

- (2) **人口类特征**:检查 Application 子类名称是否匹配已知壳程序类名模式;分析 <clinit> 和 attachBaseContext 中的字节码模式。
- (3) **多 DEX 负载特征**:检查 APK 中是否包含多个 DEX 文件及其有效类结构。
- (4) **资源文件特征**:对 assets/ 中 100KB~10MB 文件进行 Shannon 熵值分析。需要注意,JPEG、PNG、MP3 等常见媒体资源经压缩编码后字节熵值天然偏高(JPEG 通常在 7.5~7.9,PNG 在 7.0~7.8),若不加区分直接套用熵阈值会产生大量误报。因此本特征采用双条件判定:先检查文件扩展名或文件头魔数是否命中已知媒体/压缩格式(.jpg,.png,.mp3,.mp4,.webp,.ttf,.zip等);仅当文件未命中已知格式签名,且 Shannon 熵值 $H > 7.8$ 时,才标记为可疑加密 payload。
- (5) **DEX 结构完整性**:检查 checksum 与 signature 是否与文件实际内容一致。

5.4.2 混淆特征.

- (1) **标识符混淆指标**:类名/方法名长度分布的统计特征(均值、方差、短名比例);包名层次结构异常。
- (2) **反射混淆指标**:反射 API 调用密度;字符串常量池中疑似类名/方法名的字符串比例;调用图中不可解析边的比例。
- (3) **控制流混淆指标**:方法的圈复杂度分布(圈复杂度超出基线分布 P_{95} 视为可疑,阈值通过第 5.4.4 节所述的基线校准流程确定);控制流平坦化的特征模式(包含 packed-switch 或 sparse-switch 指令且目标基本块统一跳回分发器);调用间接化特征。
- (4) **字符串加密指标**:常量池中非可打印字符串占比;<clinit> 中是否包含循环异或、AES 块操作的特征指令序列。
- (5) **操作码统计特征**:256×256 操作码转移矩阵(OMM),参考 CorDroid [6] 的方法作为混淆指纹;垃圾代码密度;各操作码类别的分布比例。
- (6) **标准混淆指纹(ProGuard/R8 识别)**:Android 官方构建工具链 ProGuard 和 R8 对发布版本 APK 默认施加标识符重命名,其混淆策略具有高度可预测的规律性,必须与对抗性混淆加以区分。ProGuard/R8 的典型指纹包括:(1)包路径结构呈现 a.a.a、b.b.b 等单字符字母序前缀模式;(2)同一包内类名按字母序连续分配(a.class、b.class、c.class),形成严格的字典序流水线;(3)混淆范围限定于非人口类和非反射调用目标,因此经 ProGuard/R8 处理的应用在反射调用密度、控制流复杂度和字符串加密程度上均与未混淆基线保持在同一量级。该特征用于判定标识符缩短的可归因性:若类名缩短模式命中 ProGuard/R8 指纹,且检测范围内未发现反射替换、控制流变形或字符串加密等对抗特征,则将其归类为 standard_obfuscation,不作为对抗混淆告警。

5.4.3 环境检测特征.

- (1) **关键 API 调用扫描**:扫描 DEX 字节码中的方法调用,匹配系统属性读取(System.getProperty()、Build.FINGERPRINT 等)、文件系统探测(/proc/self/maps、/sys/qemu_trace 等)、传感器探测、调试状态检测、用户数据访问五类 API。

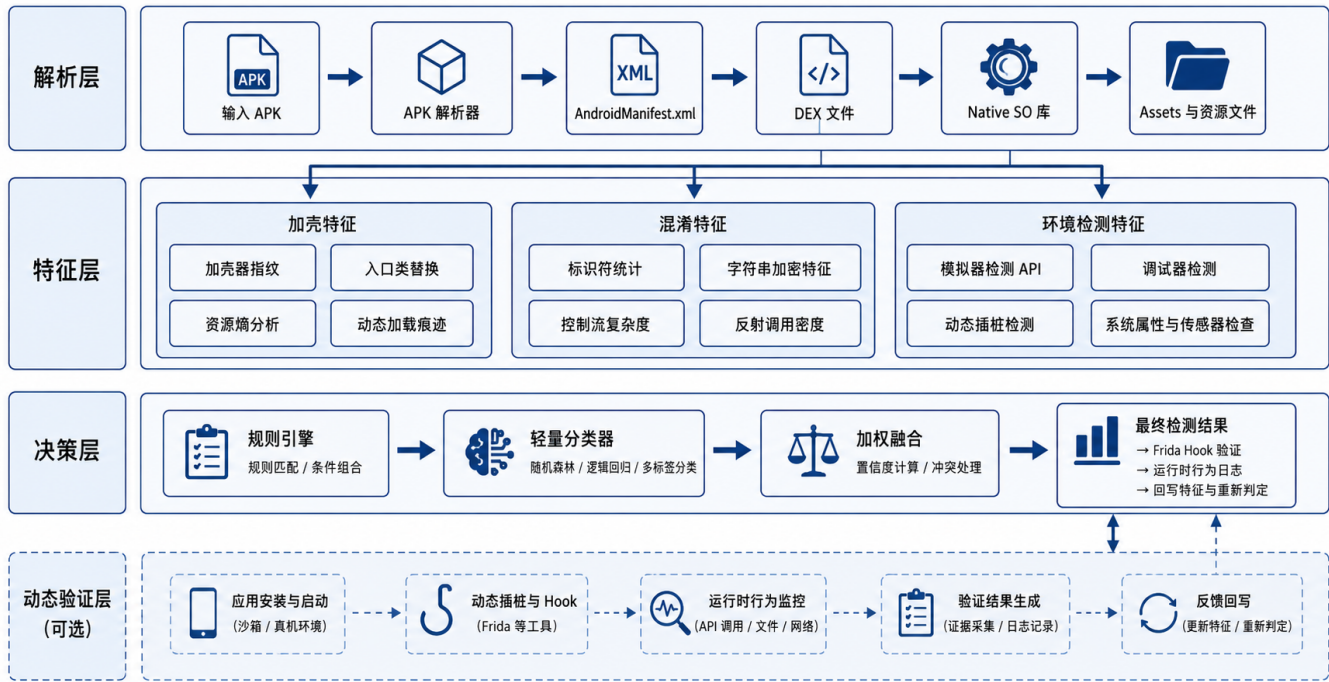


Figure 2: HardenInspector 架构概览。静态解析、特征提取和规则决策是期末 MVP 主路径；动态验证层为可选扩展。

(2) **特征字符串模糊匹配**: 在 DEX 常量池和 SO .rodata 段中搜索 /proc/self/, ptrace, TracerPid, frida, xposed, qemu, emulator 等关键词。

(3) **上下文加权**:

- 时间加权**: 调用出现在 <clinit> 或 attachBaseContext 中权重更高。
- 控制流加权**: 探测 API 的调用结果被直接用于分支决策时权重加倍。
- 组合加权**: 同时出现多种环境检测类别时获得组合加分。

需要指出的是, NDSS 2022 [9] 揭示的环境检测体系远比上述静态特征丰富——攻击者可以利用使用磨损 (照片数量、联系人数量、通话记录)、设备配置 (自动同步时间、开发者选项状态)、硬件指纹 (传感器存在性向量) 等运行时才能获取的动态特征进行多维组合判定。这些特征的共同点是, 它们在 APK 的静态文件中不留下任何痕迹, 完全依赖运行时的系统状态查询。因此, HardenInspector 的环境检测维度存在一个明确的天花板——它只能识别“代码中编写了环境探测逻辑”, 而无法判断该逻辑在真实设备上是否会产生规避效果, 也无法捕捉纯动态的环境指纹采集行为。

然而, 这一局限并不意味着环境检测维度的设计价值被削弱。首先, 静态字符串扫描与 API 调用分析足以捕获绝大多数已知的环境检测模式。NDSS 2022 中报告的模拟器属性检测、调试器检测和 DBI 框架检测均可在字节码或常量池中找到对应的证据。其次, 在多维度组合分析的视角下, 即使环境检测维度只提供了“疑似存在环境探测意图”的弱信号, 当它与加壳维度的强信号或混淆维度的中信号同时出现时, 仍能为分析者提供关键的上下文信息——一个同时使用了加壳和标识符混淆的 APK 如果在 <clinit> 中查询 ro.kernel.qemu, 其恶意思图的

可信度远高于一个仅在屏幕适配代码中调用 Build.MODEL 的正常应用。正是这种跨维度的信号协同, 使得即使每个维度都只覆盖其可观测部分, 整体分析画像仍然比单一维度更完整。

5.4.4 特征基线校准 上述混淆特征中的连续型指标 (短标识符比例、反射调用密度、圈复杂度分布、非可打印字符串占比等) 在缺乏参照的情况下无法直接判定“异常”。若直接套用经验常数 (如“圈复杂度 > 50”), 不仅忽视了 DEX 字节码与 JVM 字节码在指令密度上的系统性差异, 也使得规则在不同规模、不同功能类别的应用上缺乏一致的判定标准。

HardenInspector 采用**基线校准**的方式确定规则阈值: 在正式检测之前, 先对一组已知未加固的良性 APK 提取相同的特征向量, 以其分布的分位数作为异常阈值。具体流程如下:

- 基线样本选取**: 从 F-Droid 选取 20–30 个开源 APK, 覆盖工具类、多媒体类、网络通信类等常见应用类别, 人工确认未使用 Obfuscapk、ProGuard/R8 配置之外的混淆手段, 且不含加壳或环境检测逻辑。
- 特征分布估计**: 对每个基线样本计算混淆维度的全部连续型特征值, 得到每个特征的样本分布 $\{x_1, x_2, \dots, x_n\}$, 计算其均值 μ 、标准差 σ 和第 95 分位数 P_{95} 。
- 阈值设定**: 对于“数值越高越可疑”的指标 (如短标识符比例 p_{short} 、反射调用密度 ρ_{ref} 、圈复杂度超标比例 $\gamma_{complex}$ 、非可打印字符串占比 $r_{nonprint}$), 规则阈值设为基线分布的第 95 分位数 P_{95} 。该设定的含义是: 仅有约 5% 的正常应用会因自然波动而触发单条规则, 而组合规则和 ProGuard/R8 指纹过滤将进一步降低整体误报率。
- 阈值可调性**: 上述 P_{95} 阈值作为默认配置写入 YAML 规则文件, 分析者可根据实际场景调整分位数 (如严苛模式使用 P_{99} 、宽松模式使用 P_{90}), 规则引擎无需重新编译。

基线校准不是一次性步骤：当规则库增加新特征、或基线样本集扩充时，应重新运行校准流程以更新阈值。该机制将“可疑”从经验常数转化为数据驱动的统计判断，使规则的可解释性从证据链延伸到阈值来源。

5.5 决策层

决策层采用 **规则引擎 + 可解释评分** 的设计。期末原型不依赖训练分类器，而是优先保证每个检测结论都能追溯到静态证据。

5.5.1 规则引擎. 规则引擎维护一个可扩展的规则库，每条规则包含 rule_id、category、label、conditions、logic (AND/OR)、severity 和 explanation。规则通过外部 YAML/JSON 文件配置，与核心代码分离。规则命中后，系统记录具体证据位置，例如文件名、类名、方法名、字符串常量或字节码模式，并依据证据强度给出“强/中/弱”三级风险标记。

5.5.2 可选统计扩展. 对于规则引擎难以直接表达的连续型特征，原型只保留轻量统计评分，例如短标识符比例、反射调用密度、资源文件熵值、可疑字符串命中数量等。这些统计值不被解释为训练模型的分类概率，而是作为规则条件和报告中的辅助证据。若期末后继续扩展，可在已有特征向量基础上尝试 RandomForest、XGBoost 等轻量模型，但该部分不作为本阶段交付前提。

5.5.3 决策输出. 决策输出以多标签结果和证据链为主，而不是单一概率分数。对于每个标签，报告包含命中规则编号、证据位置、证据类型、风险等级和解释文本。

三个维度各自独立评估，维度级结论由该维度内部已命中规则的严重性共同决定。每条规则在规则库中预定义了严重性字段（强/中/弱），维度级结论的判定规则如下：

- (1) 若维度内任意严重性为“强”的规则命中，则该维度结论为强；
- (2) 若无强规则命中，但严重性为“中”的规则命中数 ≥ 2 ，则维度结论为中；
- (3) 若中规则仅命中 1 条，或弱规则命中数 ≥ 2 ，则维度结论为弱；
- (4) 若仅弱规则命中 1 条，则维度结论为需人工复核。

该设计的优点在于：维度级结论的推导过程完全由单条规则的严重性和命中计数决定，不依赖任意赋权或规则间排序，相同输入始终产生相同输出；同时每条命中的规则及其证据位置仍完整保留在报告中，分析者可以自行回溯结论来源。对于加壳维度，典型的强结论路径为 P1（已知加壳 SO 库）单条命中；中结论路径为 P2（Stub 入口类）+ P3（高熵资源文件）同时触发；弱结论路径为仅 P4（DEX 结构异常）单独命中。

在混淆维度上，决策层额外引入组合判断机制以区分标准混淆与对抗混淆。标识符缩短 (O1) 在独立命中时不直接输出对抗混淆标签，而是先比对 ProGuard/R8 标准混淆指纹：若命中则标记为 standard_obfuscation，仅当 O1 与反射替换 (O2)、控制流混淆 (O3) 或字符串加密 (O4) 中至少一条同时命中时，才输出 identifier_rename 标签。该设计的依据来自 CorDroid [6] 的实验观察：ProGuard/R8 的标准混淆仅改变标识符而不引入反射调用、控制流变形或字符串加密等对抗手段，而真正以阻碍分析为目标的混淆通常组合使用多种技术。这一判定路径保证了混淆检测不会在商业应用的正常发布流程上产生大量误报。

5.6 动态验证层

动态验证层在本项目中作为可选扩展，而非期末原型的核心依赖。其作用是在静态规则命中但证据不足，或多个规则给出冲突结论时，提供少量运行时观察。可选实现基于 Frida 框架，在 Android 模拟器中 Hook System.getProperty、Build.*、Debug.isDebuggerConnected、ClassLoader.loadClass、Method.invoke 等关键点，记录环境检测、动态类加载和反射调用等运行时行为。

动态验证层的设计借鉴了 PackerGrind [19] 的“运行—收集—回写”思路，但目标不同：本项目不尝试恢复完整 DEX，也不要求自动探索所有执行路径，而是将少量 Hook 日志作为静态证据链的补充。由于动态执行依赖模拟器配置、样本触发路径和 Frida 本身的可检测性，期末评估仍以静态规则和可复现测试样本为主。

若时间允许，动态验证层可以作为演示功能实现：对少量测试 APK 启动应用并收集 Hook 日志，展示静态检测结果如何被运行时观察补强。但该部分不作为最小可交付版本的必要功能。

5.7 规则库设计与维护

规则库是 HardenInspector 规则引擎的知识核心。表 4 给出了加壳维度的代表性规则示例。

Table 4: 加壳维度代表性规则示例

加壳器	关键 SO 库	壳程序类名模式	规则强度
360 加固 v5	libjiagu.so	StubApp	强
腾讯乐固	libtup.so	TApplication	强
爱加密	libDexHelper.so	IApplication	强
梆梆加固	libsecexe.so	WrapperApp	中
阿里聚安全	libmobisec.so	SecurityApplication	中
百度加固	libbaiduprotect.so	ProxyApplication	中
LIAPP	libqbelf.so	无固定模式	弱
DexProtector	libdexprotector.so	无固定模式	弱

在课程项目实现中，规则库优先覆盖表中列出的少量典型加壳器指纹，以及混淆和环境检测相关的通用静态特征。规则文件采用 YAML 或 Python 配置形式，与解析逻辑解耦，便于在后续实验中逐步补充新的加壳器、混淆器和环境检测模式。该设计的重点不是维护完整的商业规则库，而是保证每条检测结论都能追溯到具体文件、类名、方法名或字符串证据。

为了使期末实现目标更明确，表 5 给出了 MVP 阶段优先实现的规则清单。该清单不追求覆盖所有商业加固变体，而是覆盖课程项目中最容易构造、复现和解释的静态证据。

5.8 关键算法与实现要点

5.8.1 多维度特征提取. 下述算法给出了特征层中三个维度特征的统一提取流程：

算法：多维度特征提取

- (1) 输入：解析层 IR（Manifest AST、DEX 列表、SO 列表、assets 列表）
- (2) 输出：特征向量 $\mathcal{F} = \langle \mathbf{f}_p, \mathbf{f}_o, \mathbf{f}_e \rangle$

Table 5: HardenInspector MVP 阶段优先实现的规则清单

维度	规则编号	核心条件	输出标签
加壳	P1	SO 文件名命中 libjiagu.so、libtup.so、libsecexe.so、libDex-Helper.so 等已知加壳库	packed / packer_name
加壳	P2	Manifest 中 Application 类名命中 StubApp、WrapperApp、ProxyApplica- tion 等壳入口模式	stub_application
加壳	P3	assets/ 或 lib/ 中存在高熵大文件，且与动态加载 API 共同出现	encrypted_payload
加壳	P4	DEX checksum/signature 异常，或入口 DEX 有效类数量明显偏低	dex_hidden
混淆	O1	类名/方法名短标识符比例异常；独立命中时先识别 ProGuard/R8 指纹，若命中则标记为标准混淆；仅与 O2-O4 至少一条同时命中时才输出 identifier_rename	identifier_rename / standard_obfuscation
混淆	O2	反射 API 调用密度异常，且字符串池中存在疑似类名/方法名常量	reflection_replace
混淆	O3	方法中出现 packed-switch/sparse-switch 分发器模式或异常圈复杂度	control_flow_obfuscation
混淆	O4	常量池中非可打印字符串比例较高，或初始化路径中出现 XOR/AES 相关指令模式	string_encryption
环境检测	E1	调用 System.getProperty 或读取 Build.*，并检查 qemu、fingerprint、model 等键值	emulator_check
环境检测	E2	字符串或文件访问命中 /proc/self/maps、TracerPid、/sys/qemu_trace	process_check
环境检测	E3	出现 Debug.isDebuggerConnected、ptrace 或 JDWP 相关检测逻辑	debugger_check
环境检测	E4	字符串池或 Native 字符串命中 Frida、xposed、substrate 等关键词	hook_framework_check

- (3) 步骤 1 (加壳特征):

 - (a) 遍历 SO 文件列表，计算文件名与已知加壳器 SO 库的匹配；提取 .rodata 字符串匹配；
 - (b) 解析 Manifest 中 Application 类名，匹配壳程序模式；
 - (c) 对 assets 中 100KB-10MB 文件计算 Shannon 熵值 $H = -\sum_i p_i \log_2 p_i$ ；
 - (d) 检查 DEX 文件 checksum/signature 完整性。

(4) 步骤 2 (混淆特征):

 - (a) 遍历所有类/方法，收集类名长度分布 $\{l_1, l_2, \dots\}$ ；
 - (b) 统计反射调用密度 ρ_{ref} 、非可打印字符串比例 $r_{nonprint}$ 、垃圾代码密度 δ_{junk} ；
 - (c) 建立 256×256 操作码转移矩阵 **M** (OMM 指纹)；
 - (d) 计算每个方法的 McCabe 圈复杂度，统计 $V(G) > 50$ 的比例 $\gamma_{complex}$ 。

(5) 步骤 3 (环境检测特征):

 - (a) 扫描关键 API 调用和特征字符串，统计各类别命中次数；
 - (b) 对每个命中标记上下文权重 (时间权重 w_t : 在 `<clinit>` 中 $w_t = 2$ ；控制流权重 w_c : 紧随 `if-*` 时 $w_c = 2$)；
 - (c) 计算加权得分 $S_e = \sum_k w_t^{(k)} \cdot w_c^{(k)}$ 。

(6) 步骤 4: 返回 $\mathcal{F} = \langle \mathbf{f}_p, \mathbf{f}_o, \mathbf{f}_e \rangle$ 。

5.8.2 运行时性能优化. HardenInspector 在静态分析阶段的性能优化策略按实现优先级分为核心策略和可选策略：

基础缓存：核心实现至少缓存 APK 解包目录和已解析的文件列表，避免在一次扫描流程中重复读取同一文件。更细粒度的 SHA-256 特征缓存适合后续批量扫描版本，不作为期末原型的必要功能。

分阶段特征提取：核心实现按加壳、混淆、环境检测三个维度顺序提取特征，保证日志和证据链易于调试。并行化可以作为后续优化，但不影响原型的功能完整性。

DEX 解析范围控制：对于大型多 DEX 应用，优先解析入口 DEX (`classes.dex`) 和 Manifest 指向的 Application 相关代码。若时间有限，其余 DEX 可以先做字符串与类名级扫描，避免在课程原型中引入过重的全量字节码分析。

基于签名的快速路径：对于 SO 库名称与已知加壳器高度匹配 (编辑距离 ≤ 2) 的 APK，优先输出加壳器相关证据并标记为“强规则命中”。这一快速路径可以减少不必要的深度统计，但不会跳过混淆和环境检测的基础扫描，以免遗漏同一 APK 中并存的其他加固信号。

5.9 输出格式

HardenInspector 的输出为结构化的 JSON 报告和可读的摘要文本。表 6 给出了一个示例输出。

Table 6: HardenInspector 输出示例

维度	检测结果
加壳	360 加固 v5 (risk: strong) Evidence: libjiagu.so (SO hash match); StubApp in manifest; assets/ijm_lib/*
混淆	identifier_rename (risk: strong); string_encryption (risk: medium) Evidence: 87% classes with 1-2 char names; non-printable strings in constant pool
环境检测	present (risk: medium) Evidence: Build.FINGERPRINT in <clinit>; /proc/self/maps string in .rodata
汇总	3 个维度均存在可解释证据，建议人工复核

6 评估计划

6.1 数据集构建

本项目评估数据集以可复现的小规模样本为主,不依赖商业数据源或大规模恶意样本平台。数据集分为以下几类:

- (1) **良性对照组 (兼基线校准)**: 从 F-Droid 选取 20~30 个开源 APK, 并额外构建 3~5 个简单测试 APK。该组样本同时承担两项职能: (a) 作为未加固基线, 参与第 5.4.4 节所述的阈值校准流程, 为混淆维度的连续型特征建立正常分布参照; (b) 作为误报分析对照组, 用于评估规则在正常应用上的误触发情况。
- (2) **加壳样本组**: 优先选择可公开获取或便于实验的 2~3 种加固工具/服务, 对简单测试 APK 进行处理; 若商业加固器获取受限, 则将加壳检测评估限定为可观测规则验证, 例如 Stub Application、可疑 Native 库、动态加载 API 和高熵资源文件。
- (3) **混淆样本组**: 使用 Obfuscapk [1] 或 ProGuard/R8 对 10~20 个 APK 施加标识符重命名、字符串加密、反射替换、控制流扰动等有限几类变换, 构造可控的单一混淆和组合混淆样本。
- (4) **环境检测样本组**: 编写 5~10 个小型测试 APK, 分别包含模拟器属性检查、调试器检测、Frida/Xposed 字符串探测、/proc/self/maps 扫描等典型逻辑, 用于验证环境检测规则是否能够命中。

表 7 将上述数据集进一步拆分为可执行的实验单元, 明确每类样本的标签来源和验证目标。

Table 7: 期末 MVP 评估数据集与验证目标

样本类型	数量	标签来源	验证目标
F-Droid 良性 APK	20~30	开源应用, 人工确认无主动加固	估计误报来源
自写基线 APK	3~5	自行编写, 保留源码	验证解析与报告格式
加壳测试 APK	2~3 组	加固前后对照或可观测规则	验证 P1~P4 规则
可控混淆 APK	10~20	Obfuscapk/ProGuard 配置	验证 O1~O4 规则
环境检测 APK	5~10	自写检测逻辑与源码位置	验证 E1~E4 规则

6.2 评估指标

对每个维度分别报告: (1) 加壳识别: 规则命中率、误报样例和漏报样例; (2) 混淆识别: 各类混淆特征的命中情况, 以及在可控样本上的 Precision/Recall; (3) 环境检测识别: 典型检测 API 和字符串模式的覆盖率; (4) 效率指标: 单 APK 平均扫描时间和输出报告大小; (5) 解释性指标: 每个输出标签是否包含 rule_id、证据位置和人工可读解释。由于数据集规模有限, 本文不把大规模分类准确率作为期末阶段的主要目标, 而更关注检测器是否能输出可解释的证据链。

6.3 基线方法

选用以下轻量级基线作为比较:

- **文件名与字符串匹配基线**: 仅基于 Native 库名称、Stub 类名和关键字字符串进行检测。

- **特征统计基线**: 仅使用标识符长度、字符串熵、反射调用次数等统计特征, 不使用组合规则。
- **完整规则引擎**: 使用本文设计的多维特征和证据链输出, 与前两类基线比较解释性和覆盖范围。

6.4 预期结果与假设

基于对现有文献的分析, 我们将期末实现目标限定为一个一周内可完成的静态检测器原型, 并提出以下可验证假设:

静态优先假设: 对于常见加固器和教学测试样本, 仅依靠 Native 库名称、Manifest 入口类名、动态加载 API、资源文件熵值等静态特征, 就可以识别出一部分明显的加壳痕迹。该假设的目标不是完整脱壳, 而是为安全分析者提供快速预筛和证据定位。

互补性假设: 单类特征只能反映某一类混淆现象, 例如标识符长度分布适合发现重命名, 字符串熵适合发现字符串加密, 反射调用密度适合发现调用间接化。组合这些轻量特征可以比单一字符串匹配基线提供更完整的混淆画像。

可解释性假设: 检测结果如果同时给出命中的文件、类名、方法名、字符串和规则编号, 即使覆盖面有限, 也比单纯输出“疑似加固”更适合作为课程项目和后续人工分析的基础。

6.5 实验设计

为验证原型系统的有效性, 设计以下小规模实验:

- (1) **规则覆盖实验**: 统计每类样本命中的规则数量和证据类型, 检查是否能覆盖加壳、混淆、环境检测三个维度。
- (2) **误报分析实验**: 在未加固 F-Droid 样本上运行检测器, 人工检查误报来源, 例如正常动态加载、正常反射调用或合法传感器 API 使用。
- (3) **对比实验**: 与文件名/字符串匹配基线比较, 说明多维规则是否能发现更多可解释线索。
- (4) **效率实验**: 记录不同大小 APK 的扫描时间, 验证工具能否在普通笔记本上完成秒级到十秒级分析。

6.6 与工业实践的对比定位

为明确 HardenInspector 在现有安全生态中的定位, 表 8 将其与当前主流的工业实践方案进行了对比。

Table 8: HardenInspector 与现有方案的对比

方案	分析方法	输出	速度/样本	可部署性
VirusTotal 多引擎	多引擎混合	恶意/良性标签	分钟级 (网络)	高
APK 纯规则扫描	静态签名匹配	加壳器名称	秒级	高
沙箱动态分析	动态行为监控	行为报告	分钟~小时级	中
Happer [20]	硬件追踪	脱壳后 DEX	分钟级	低 (需硬件)
HardenInspector	静态规则为主	多标签 + 证据链	秒~十秒级	高

HardenInspector 的定位不是替代工业级沙箱或商业加固识别服务, 而是在课程项目范围内实现一个轻量、可解释、可复现的静态预筛工具。它的价值在于帮助分析者快速定位 APK 中

可能与加固相关的证据。在完整实现后,该思路可以扩展到以下场景:

批量样本预筛:对一组课程实验样本或公开 APK 进行静态扫描,快速筛选出使用了动态加载、可疑 Native 库或环境检测逻辑的样本。

安全研究中的样本理解:在手工逆向或动态分析之前,先了解样本中是否存在标识符混淆、字符串加密、反射调用、Frida/Xposed 探测等信号,从而决定后续分析路径。

防御性自检:开发者可以用相同思路检查自己的测试 APK 是否包含预期的混淆或加壳痕迹,但这一场景不作为本课程项目的主要评估目标。

6.7 典型应用场景分析

本节以三个具有代表性的加固场景为例,展示 HardenInspector 在不同情况下的预期检测行为。

6.7.1 场景一: 360 加固 v5 保护的金融应用. 输入特征: APK 中 lib/armabi/libjiagu.so 匹配已知 360 加固 SO 库名称; AndroidManifest.xml 中 Application 类名为 com.qihoo.util.StubApp; assets/ 中包含具有加密数据特征的 ijm_lib/ 目录; classes.dex 的 checksum 与内容不匹配; 混淆方面,短类名比例较高,并出现反射调用和异常操作码分布。

预期输出:加壳: 360 加固 v5 (强规则命中); 混淆: identifier_rename (强)、string_encryption (中)、reflection_replace (中); 环境检测: present (中)。报告列出对应 SO 文件、Manifest 入口类、assets 目录和 DEX 字符串/调用证据。

决策路径:规则引擎首先命中 360 加固相关强规则,随后继续执行基础混淆和环境检测扫描。混淆与环境检测结论来自短标识符比例、反射调用、特征字符串和上下文位置等静态证据,不依赖训练分类器。

6.7.2 场景二: Obfuscapk 生成的组合混淆测试应用. 输入特征:无明显加壳特征; 混淆方面,类名长度显著变短,部分方法包含复杂控制流,反射调用和疑似垃圾代码比例高于未混淆基线。

预期输出:加壳: not_packed (未发现明显证据); 混淆: identifier_rename (强)、control_flow_flattening (中)、junk_code_insertion (中); 环境检测: not_present (未发现典型证据)。对于字符串加密,如果只出现少量非可打印字符串,则标为“需人工复核”而非直接给出确定标签。

决策路径:规则引擎未命中加壳器规则,因此加壳维度只输出“未发现明显加壳证据”。混淆维度由短标识符、复杂控制流和疑似垃圾指令等规则共同触发,报告中保留命中位置,便于人工确认。

6.7.3 场景三: 含环境检测的商业间谍软件. 输入特征:加壳层面: 仅检测到梆梆加固的基础特征 (libsecexe.so 存在但无高级反调试逻辑); 混淆层面: 标识符混淆程度中等 (45% 短类名); 环境检测层面: System.getProperty() 调用出现在 <clinit> 中, 参数包含 ro.product.model、ro.build.fingerprint、ro.kernel.qemu; 同时存在 /proc/self/maps 字符串引用和 Debug.isDebuggerConnected() 调用。

预期输出:加壳: 梆梆加固基础特征 (中); 混淆: identifier_rename (弱/中); 环境检测: present (强)。环境检测报告应列出系统属性键、/proc/self/maps 字符串和调试状态检测调用的位置。

决策路径:环境检测特征在此样本中信号最强,多个系统属性、进程文件和调试检测证据同时出现,且部分调用位于早期

初始化路径。该样本体现了 HardenInspector 的设计重点: 环境检测并非加壳的专利,它可以独立存在或与轻度加壳组合出现。只有同时分析三个维度才能给出更完整的加固技术画像。

6.8 工程实现路线图

考虑到期末实现周期约一周, HardenInspector 的工程实现按“核心功能优先、动态能力可选”的方式推进:

第一阶段 (1-2 天):实现 APK 解析和基础特征提取。基于 Androguard [4] 或轻量解包逻辑读取 Manifest、DEX 字符串、类名、方法名、Native 库名称和资源文件列表; 实现资源文件熵值、标识符长度分布、可疑字符串匹配等基础统计。

第二阶段 (2-3 天):实现规则引擎和报告输出。将检测规则组织为可维护的 YAML 或 Python 配置,覆盖加壳、混淆、环境检测三类证据; 输出 JSON 报告和可读摘要,报告中包含命中的规则、证据位置和风险解释。

第三阶段 (1-2 天):构建小规模测试集并完成评估。使用 F-Droid、自写 APK、Obfuscapk/ProGuard/R8 处理样本验证规则覆盖和误报情况; 整理典型输出案例。若时间允许,再实现少量 Frida Hook 脚本作为动态验证的扩展演示,但不将其作为核心交付依赖。

7 讨论与未来工作

7.1 当前方案的局限性

对 Native 混淆的覆盖不足:当前的混淆特征提取主要集中在 DEX 字节码层面。但现代加固方案已大量使用 Native 层混淆,通过 Obfuscator-LLVM 等工具对 C/C++ 代码施加控制流平坦化、指令替换、虚假控制流等混淆后编译为 .so 库,再通过 JNI 与 Java 层交互。这些 Native 层混淆在 DEX 字节码层面完全不可见,唯一的可观测特征存在于 .so 文件的二进制结构中,如异常的控制流图模式、频繁的分发器跳转、虚假分支注入等。当前 HardenInspector 对 Native 层的分析仅停留在符号表扫描和字符串匹配阶段,远不足以检测 Native 层混淆。

为了解决这一问题,未来版本可集成 Ghidra 或 Binary Ninja 的脚本化反汇编能力: 优先分析 JNI 导出函数和可疑导入函数,在 Native 层提取字符串、导入表、控制流复杂度等轻量证据。该扩展的挑战在于单个大型 .so 文件的完整反汇编和 CFG 构建可能需要数分钟,因此不适合作为期末 MVP 的默认功能。

数据集偏差与泛化:期末阶段的数据集主要来自 F-Droid、自写 APK、Obfuscapk [1] 或 ProGuard/R8 生成的可控混淆样本,以及少量可获得的加固样本。这一数据集构建策略存在两个偏差源: (1) **工具偏差:**教学样本和开源混淆工具的实现方式可能与真实商业加固器或攻击者使用的混淆工具存在差异; (2) **规模偏差:**小规模样本只能验证规则可行性,不能证明检测器已经覆盖真实 Android 生态中的全部加固变体。

为了缓解这一问题,期末实现阶段将优先保证规则和样本来源可复现: 所有测试 APK、规则文件和检测输出均随项目提交; 商业加固器和 VirusTotal 样本仅作为后续扩展方向,不作为课程项目的必要依赖。

对抗性加固:如果加固工具开发者了解 HardenInspector 的特征提取策略,他们可能设计针对性的对抗措施。例如: (1) 在 Native 库命名上伪装为系统库,如将 libjiagu.so 重命名为 libandroid_runtime.so,使用通用的壳程序类名替代已知模式; (2) 在字符串常量中插入误导性关键词,如在 .rodata 段中故意包含 /proc/self/maps 等环境检测关键词; (3) 在 DEX 中注入虚假的混淆特征,如大量 NOP 指令和异常复杂的方法结构。规则式检测的优势是解释性强、实现成本低,但面对刻意

伪装时容易产生误报或漏报。因此,期末原型应明确输出“证据链”和“需人工复核”的中间状态,而不是声称能够稳健对抗所有加固变体。

Android 版本演进: Android 平台的持续演进可能改变加固技术的可观测特征。

多 DEX 与动态加载的边界情况: 部分大型应用使用合法的 MultiDex 机制分割代码; React Native、Flutter 等框架将业务逻辑打包在非标准格式中。区分“合法的动态加载”和“加壳器的恶意隐藏”需要更细粒度的语义分析。

加固技术的中性本质带来的伦理考量: HardenInspector 输出的加固技术清单可以被安全分析师用于快速筛选可疑应用,但也可能被恶意软件作者用于“对抗性评估”。工具的公开需要在“赋能防御”和“不辅助攻击”之间找到平衡点。

7.2 与现有工具生态的集成

HardenInspector 的设计目标之一是与现有工具链无缝集成:

与 Androguard 的协作: 解析层直接复用 Androguard [4] 的 DEX 解析和 Manifest 分析能力。Androguard 提供的 Python API 为特征提取提供了成熟的编程接口。通过复用这一经过广泛验证的逆向分析框架, HardenInspector 避免了对底层 DEX 格式的重复实现,将开发重点聚焦在加固技术特有的特征工程和决策逻辑上。

与公开样本标签的互补: 在后续扩展中, HardenInspector 的输出可以与 VirusTotal、AndroZoo 等公开样本标签结合使用。但在课程项目中,我们更关注本地可复现的静态证据提取,不依赖外部在线服务。

与 Frida 生态的衔接: Frida Hook 脚本可作为后续扩展模块,用于复核少量静态规则命中的样本。期末阶段若时间允许,可实现 `System.getProperty()`、`Debug.isDebuggerConnected()`、`ClassLoader.loadClass()` 等少量 Hook 点,展示运行时日志如何补充静态证据;但该部分不作为核心交付和评估依据。

7.3 未来改进方向

(1) 混淆溯源: 当前方案输出的是混淆技术类别,未来可进一步实现混淆工具链溯源,通过分析不同混淆器在字节码生成模式上的细微差异,识别具体的混淆工具及其版本。工具链溯源能力在实际应用中也具有重要价值,例如检测到某应用中使用了已知恶意软件家族偏好的特定混淆工具链版本,该信息可作为威胁情报的重要组成部分。

(2) 加固深度分级: 借鉴 PackerGrind [19] 中对加壳器“保护层数”的刻画以及 Happer [20] 对加壳行为模式的分类,未来可将当前“强/中/弱”规则结果扩展为更细的加固深度分级。该方向需要更多标注样本和人工校准,因此不纳入期末阶段的核心目标。

(3) 增量分析优化: 批量扫描场景中,大量应用共享相同的框架库和第三方 SDK。例如,使用 Unity 引擎的游戏 APK 共享相同的 `libil2cpp.so` 和 `libunity.so`;使用 React Native 的应用共享相同的 JavaScript 引擎 `so` 库。后续版本可通过构建共享库的特征指纹缓存,避免对相同组件重复提取特征。

(4) 语义辅助分析: 当前的上下文加权规则在区分“正常使用传感器 API”和“为了检测沙箱而使用传感器 API”时可能产生误报。例如,一个健康类应用在初始化阶段检查传感器可用性属于正常行为,但规则可能将其标记为环境检测。未来可以结合更细粒度的数据流分析或少量人工复核流程,对边界样本进行语义确认。大语言模型也可作为辅助解释工具探索,但不应作为期末原型的核心依赖。

(5) 规则库扩展: 将检测逻辑与规则文件解耦,使后续可以逐步补充新的加壳器、混淆器和环境检测模式,而不需要重写核心解析逻辑。

8 结论

Android 应用的加固技术已从简单的标识符混淆演进为多层次、多机制的对抗体系,给移动安全分析带来了持续挑战。本文归纳了覆盖代码混淆、运行时加壳和环境感知检测三个维度的统一分类体系,明确了现有研究在技术识别这一特定任务上的能力空白。

通过系统综述,我们总结出三条核心发现:(1) 现有 ML 检测方法在“恶意/良性”分类上取得了优异性能,但其模型输出无法直接转化为加固技术标签,存在识别粒度的结构性缺失;(2) 现有 unpacking 系统在动态 DEX 恢复方面取得了显著进展,但其对动态执行的强依赖限制了大规模部署的可行性,且对脱壳后产物的混淆鉴别缺乏关注;(3) 环境检测研究证明了沙箱可被可靠识别,但缺乏反向检测能力,无法识别一个 APK 是否正在主动探测分析环境。这三条发现共同指向了同一个研究缺口:目前缺乏一个能统一识别混淆、加壳、环境检测三类加固技术的轻量级工具。

基于发现,本文提出了一个以静态规则分析为主、动态验证为可选扩展的 APK 加固技术检测框架 HardenInspector。其核心创新在于:(1) 将任务目标从“恶意检测”转向“技术识别”,输出可解释的多标签结果及证据链;(2) 采用规则引擎组织加壳、混淆和环境检测三类静态证据,便于在课程项目周期内完成原型实现;(3) 将训练分类器、深度 Native 分析和动态 Hook 复核放在后续扩展方向,而不是期末交付的必要前提。框架的分层架构在保证功能完整性的同时,实现了模块间的解耦,便于独立测试、替换和扩展。

在期末阶段,我们将完成 HardenInspector 的课程项目原型:输入 APK 文件,解析 Manifest、DEX、Native 库和资源文件,提取加壳、混淆、环境检测相关特征,并输出包含命中规则和证据位置的 JSON 报告。评估将基于 F-Droid、自写测试 APK 和可控混淆样本,重点验证规则覆盖、误报来源和运行效率。若时间允许,将补充少量 Frida Hook 脚本作为动态验证演示,但核心交付仍是可复现、可解释的静态检测器。

References

- [1] Simone Aonzo, Gabriel Claudiu Georgiu, Luca Verderame, and Alessio Merlo. 2020. Obfuscapk: An Open-Source Black-Box Obfuscation Tool for Android Apps. *SoftwareX* 11 (2020), 100403.
- [2] Daniel Arp, Michael Spreitzerbarth, Malte Hübner, Hugo Gascon, and Konrad Rieck. 2014. Drebin: Effective and Explainable Detection of Android Malware in Your Pocket. In *Proceedings of the Network and Distributed System Security (NDSS) Symposium*.
- [3] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traou, Damien Octeau, and Patrick McDaniel. 2014. FlowDroid: Precise Context, Flow, Field, Object-sensitive and Lifecycle-aware Taint Analysis for Android Apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 259–269.
- [4] Anthony Desnos and Geoffroy Gueguen. 2011. Androguard: Reverse Engineering and Malware Analysis for Android Applications. <https://github.com/androguard/androguard>.
- [5] Yue Duan, Mu Zhang, Abhishek Vasisht Bhaskar, Zhichen Zhao, Heng Yin, and Xiaojun Lin. 2018. DroidUnpack: A Generic Unpacking Framework for Android Malware. In *Proceedings of the Network and Distributed System Security (NDSS) Symposium*.
- [6] Chao Gao, Ming Cai, Shuai Yin, Guowei Huang, Hui Li, Wei Yuan, and Xiapu Luo. 2023. Obfuscation-Resilient Android Malware Analysis Based on Complementary Features. *IEEE Transactions on Information Forensics and Security* 18 (2023), 5056–5070.
- [7] Joshua Garcia, Mahmoud Hammad, and Sam Malek. 2018. Lightweight, Obfuscation-Resilient Detection and Family Identification of Android Malware.

- In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*. Journal version in *ACM Transactions on Software Engineering and Methodology*, vol. 27, no. 3, 2018.
- [8] Michael I. Gordon, Deokhwan Kim, Jeff H. Perkins, Limei Gilham, Nguyen Nguyen, and Martin Rinard. 2015. Information Flow Analysis of Android Applications in DroidSafe. In *Proceedings of the Network and Distributed System Security (NDSS) Symposium*.
 - [9] Brian Kondracki, Babak Amin Azad, Najmeh Miramirkhani, and Nick Niki-forakis. 2022. The Droid is in the Details: Environment-aware Evasion of Android Sandboxes. In *Proceedings of the Network and Distributed Systems Security (NDSS) Symposium*.
 - [10] Martina Lindorfer, Matthias Neugschwandtner, Lukas Weichselbaum, Yanick Fratantonio, Victor van der Veen, and Christian Platzer. 2014. ANDRUBIS – 1,000,000 Apps Later: A View on Current Android Malware Behaviors. In *Proceedings of the 3rd International Workshop on Building Analysis Datasets and Gathering Experience Returns for Security (BADGERS)*. 3–17.
 - [11] Davide Maiorca, Davide Ariu, Igino Corona, Marco Aresu, and Giorgio Giacinto. 2015. Stealth Attacks: An Extended Insight into the Obfuscation Effects on Android Malware. *Computers & Security* 51 (2015), 16–31.
 - [12] Enrico Mariconti, Lucky Onwuzurike, Panagiotis Andriotis, Emiliano De Cristofaro, Gordon Ross, and Gianluca Stringhini. 2017. MaMaDroid: Detecting Android Malware by Building Markov Chains of Behavioral Models. In *Proceedings of the Network and Distributed Systems Security (NDSS) Symposium*.
 - [13] Simone Mutti, Yanick Fratantonio, Antonio Bianchi, Luca Invernizzi, Jacopo Corbetta, Dhilung Kirat, Christopher Kruegel, and Giovanni Vigna. 2015. BareDroid: Large-Scale Analysis of Android Apps on Real Devices. In *Proceedings of the 31st Annual Computer Security Applications Conference (ACSAC)*. 71–80.
 - [14] Zhenyu Ning and Fengwei Zhang. 2018. DexLego: Reassembleable Bytecode Extraction for Aiding Static Analysis. In *Proceedings of the 48th IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. 249–260.
 - [15] Guillermo Suarez-Tangil, Santanu Kumar Dash, Mansour Ahmadi, Johannes Kinder, Giorgio Giacinto, and Lorenzo Cavallaro. 2017. DroidSieve: Fast and Accurate Classification of Obfuscated Android Malware. In *Proceedings of the 7th ACM Conference on Data and Application Security and Privacy (CODASPY)*. 309–320.
 - [16] Fengguo Wei, Sankardas Roy, Xinming Ou, and Robby. 2014. Amandroid: A Precise and General Inter-component Data Flow Analysis Framework for Security Vetting of Android Apps. In *Proceedings of the 21st ACM Conference on Computer and Communications Security (CCS)*. 1329–1341.
 - [17] Michelle Y. Wong and David Lie. 2018. Tackling Runtime-based Obfuscation in Android with TIRO. In *Proceedings of the 27th USENIX Security Symposium*. 1247–1262.
 - [18] Yueming Wu, Xiaodi Li, Deqing Zou, Wei Yang, Xin Zhang, and Hai Jin. 2019. MalScan: Fast Market-Wide Mobile Malware Scanning by Social-Network Centrality Analysis. In *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 139–150.
 - [19] Lei Xue, Hao Zhou, Xiapu Luo, Le Yu, Di Wu, Yajin Zhou, and Xiaobo Ma. 2022. PackerGrind: An Adaptive Unpacking System for Android Apps. *IEEE Transactions on Software Engineering* 48, 2 (2022), 551–570.
 - [20] Lei Xue, Hao Zhou, Xiapu Luo, Yajin Zhou, Yang Shi, Guofei Gu, Fengwei Zhang, and Man Ho Au. 2021. Happer: Unpacking Android Apps via a Hardware-Assisted Approach. In *Proceedings of the 42nd IEEE Symposium on Security and Privacy (S&P)*.
 - [21] Lok Kwong Yan and Heng Yin. 2012. DroidScope: Seamlessly Reconstructing the OS and Dalvik Semantic Views for Dynamic Android Malware Analysis. In *Proceedings of the 21st USENIX Security Symposium*. 569–584.
 - [22] Wenbo Yang, Yuanyuan Zhang, Juanru Li, Junliang Shu, Bodong Li, Wenjun Hu, and Dawu Gu. 2015. AppSpear: Bytecode Decrypting and DEX Reassembling for Packed Android Malware. In *Proceedings of the 18th International Symposium on Research in Attacks, Intrusions, and Defenses (RAID)*. 359–381.
 - [23] Yueqian Zhang, Xiapu Luo, and Haoyang Yin. 2015. DexHunter: Toward Extracting Hidden Code from Packed Android Applications. In *Proceedings of the 20th European Symposium on Research in Computer Security (ESORICS)*. 293–311.